LLDD BE - Job 6 ExportImpactStoreToSTA

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. บทนำ

1.1 วัตถุประสงค์

ซิงก์สถานะ + ส่งค่าชดเชยไป STA: รัน 10 mutation ตามลำดับบนตารางสถานะ ตรวจความครบของคะแนน QSSI 6 หมวด สร้างชุดสถานะที่ส่งออกได้ แล้ว publish message ไป RabbitMQ ให้ระบบ Statement (STA) รับต่อ (มติ 2026-08-24 — เลิกเขียนไฟล์ FRBC0001 + SFTP) · เนื้อข้อมูลยังเป็นสัญญาเดิม 14 ฟิลด์ แต่เป็น JSON UTF-8 ไม่ใช่ text windows-874 · ใช้ transactional outbox แบบเดียวกับ Job 4 — DB transaction ครอบ sync + outbox แล้วค่อย publish นอก transaction

1.2 ขอบเขตงาน

- Main class/script: fgi.main.ExportImpactStoreToFS / FGI_ExportImpactStoreToSTA.sh
- Phase: D
- Output: RabbitMQ message sgi_impact_store (exchange sgi.interface)
- Estimate: 24 ชั่วโมง
- Argument รับผ่าน `INPUT` (JSON) — local ใช้ env `JOB_NAME/INPUT` · AWS Batch ใช้ `argv[3]/argv[2]` · ดูหัวข้อ 5.95 · ไม่มีตาราง job_configs และไม่มีหน้าจอควบคุม (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- ตารางเวลาตั้งที่ AWS Batch scheduled event (repo ไม่มี `@Cron`) — ยกเว้น job ที่เป็น event-driven (ดูหัวข้อ Config Schema ของ job นั้น) ซึ่งห้ามตั้ง schedule · ทุก job ถูกบันทึกลง `integration_log` โดย `main.ts` อัตโนมัติ + structured log `BATCH_START/BATCH_END` พร้อม `runId`

1.3 ข้อมูลส่งมอบ

รายการ	รายละเอียด
Track	BE
Estimate	32 ชั่วโมง = implementation 24 + unit test 8 (30%)
Owner	Aphiwit <Bank> Khammoon
Legacy class เดิม	`ExportImpactStoreToFS` (`fcsJar/src/th/co/gosoft/fgi/main/ExportImpactStoreToFS.java`)
Target repository	`SBP/srm-sps-spsap-sop-sgi-batch` (NestJS 11 + TypeORM · schema `sps_store` · มติ 2026-09-02 — ย้ายมาจาก store-backend) — batch runner ของ SBP ที่รันอยู่แล้ว 42 job บน AWS Batch · ลงทะเบียน job ใน `src/main.ts` แล้วรับ argument ผ่าน `JOB_NAME/INPUT` (local) หรือ `argv[3]/argv[2]` (AWS Batch) · ไม่ผ่าน BFF และไม่เปิด HTTP · ตารางเวลาเป็น AWS Batch scheduled event ไม่ใช่ `@Cron` · ดู `srm sps spsap sop sgi batch`

1.4 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success, data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (`x-api-key` / `x-user-*`) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Job-2-ImportImpactStore.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Program Specification

2.1 การลงทะเบียน job และ Arguments (sop-sgi-batch)

งานนี้ลงทะเบียนเป็น job ชื่อ `sgi-export-impact-store-to-sta` ใน `src/main.ts` ของ `SBP/srm-sps-spsap-sop-sgi-batch` โดยใช้ dispatcher เดิมของ repo (ไม่สร้าง runner ใหม่) — `main.ts` อ่านชื่อ job และ input มา 2 ทาง แล้วเลือกอัตโนมัติจากการมี `argv[3]` หรือไม่

ช่องทางรัน	ชื่อ job มาจาก	input มาจาก	ตัวอย่าง
Local / CLI / runbook	env `JOB_NAME`	env `INPUT` (JSON string)	`JOB_NAME=sgi-export-impact-store-to-sta INPUT='{"processDate":"2026-06-16"}' npm run start`
AWS Batch (ตารางเวลาจริง)	`process.argv[3]`	`process.argv[2]` (JSON string)	`node dist/main.js '{"processDate":"2026-06-16"}' sgi-export-impact-store-to-sta`

ตารางเวลาไม่ได้อยู่ในโค้ด — repo นี้ไม่มี `@Cron`/`@Interval` แม้แต่จุดเดียว (แม้ติดตั้ง `@nestjs/schedule` ไว้) cron ในหัวข้อ 5 เป็น นิยามของ AWS Batch scheduled event ที่ต้องตั้งตอน deploy ไม่ใช่ค่าที่อ่านจาก config file

Argument ที่รับได้ (`INPUT` เป็น JSON object · ไม่ส่ง = `{}`)

ระบบเดิมรับ argument แบบนี้: `args[0]` = yyyyMMdd (วันประมวลผล) · ไม่ส่ง = วันปัจจุบัน · parse ไม่ได้ = log `Error PARAMETER is not format` แล้วจบแบบ FAIL
ของใหม่เปลี่ยนจาก positional string เป็น JSON เพื่อให้เพิ่มฟิลด์ได้โดยไม่พังของเดิม แต่
ต้องรองรับทุกอย่างที่ระบบเดิมรับได้เป็นอย่างน้อย — ห้ามลดความสามารถลง

Field	ชนิด	ไม่ส่งแล้วได้อะไร (default)	Validation	ตรงกับ argument เดิม
`processDate`	string	วันนี้	`YYYY-MM-DD` (ค.ศ.) · parse ไม่ได้ = `INVALID_JOB_INPUT`	`args[0]` (yyyyMMdd)
`dryRun`	boolean	`false`	`true` = อ่าน/คำนวณครบแต่ไม่ commit และไม่ publish/ไม่ส่งไฟล์	ของกลาง ทุก job
`limit`	number	`null`	จำกัดจำนวนรายการที่ประมวลผลรอบนี้ (ใช้ตอน smoke test)	ของกลาง ทุก job

กติกาการ validate ที่ทุก job ต้องทำเหมือนกัน — parse `INPUT` ไม่สำเร็จ หรือฟิลด์ไม่ผ่าน validation ให้ log `BATCH_END` ด้วย `batchStatus: 'FAILED'` แล้ว `exit(1)` ก่อนแตะฐานข้อมูล (ห้าม fallback ไปค่า default เงียบ ๆ เมื่อผู้ใช้ตั้งใจส่งค่ามาแล้วผิด) · ฟิลด์ที่ไม่รู้จักให้ log warn แล้วข้าม ไม่ทำให้ job ล้ม

- `processDate` ไม่ได้เป็นแค่ label — มันเปลี่ยนผลลัพธ์ 2 ทาง: (1) dd ของมันตัดสินว่าจะสร้างชุด `I` (Initial) ให้ STA หรือไม่ (2) เดือนก่อนหน้าของมันคืองวดที่ใช้ตรวจ QSSI · ดูหัวข้อเงื่อนไขตัดสิน

2.2 Input / Output

Stage	Contract for implementation
Input	Approved/initial compensation data from FGI impact/new-store tables plus QSSI score lookup and FS export configuration.
Progress	query rows for FS, generate compensation interface payload, insert/update compensate records, upload/export, backup, notify.
Output	FS outbound data and FGI compensation tables synchronized; run summary includes exported counts and file/status.

2.3 ขั้นตอนการทำงาน

Step	Description
1	เริ่ม
2	รัน 10 mutation ตามลำดับ บน sgi_fgi_impact_processes และ sgi_fgi_impact_stores (state sync ก่อน export — ตรวจครบทั้ง 10 ขั้นตอน post-run)
3	QSSI ครบ 6 หมวด? (งวด max เดียว ในกรอบ 3 เดือน) | No: ข้ามเส้นทาง INIT — สาย APPROVE ยังไปต่อ (หมวด 8, 9, 12, 1, 10, 16 จากคอลัมน์ category ของ fcs_qssi_score)
4	สร้างชุดสถานะส่งออก: A, N, S (+ I, C เมื่อวันที่ ≥ 7 และ QSSI ครบ + Z ค้าง) (Z จะถูกแปลงเป็น S เฉพาะใน payload ที่ส่งออก — ใน DB ยังเป็น Z)
5	มีแถวส่งออก? | No: จบการทำงาน
6	ประกอบ payload JSON UTF-8 14 ฟิลด์ (วันที่ พ.ศ. ตามสัญญาเดิม) (ฟิลด์ผิด/แปลงวันที่ไม่ได้แม้แถวเดียว = ยกเลิกทั้งรอบ (คงพฤติกรรม mapData เดิม))
7	insert outbox: I,C → COMPENSATE_INIT_I/N · A,N,S,Z → COMPENSATE_APPROVE_I/N (direction = OUT · status = READY) (อยู่ใน transaction เดียวกับ 10 mutation — commit แล้วข้อมูลจะไม่หาย แม้ broker ล่ม)
8	publish ไป RabbitMQ exchange sgi.interface (routing sta.compensation.result) (นอก DB transaction · persistent + publisher confirm + mandatory)
9	ได้ publisher confirm? | No: คง outbox เป็น READY/FAILED_RETRY ให้ dispatcher ส่งซ้ำ — ไม่ rollback การ sync (เลี่ยง dual-write: การ sync สถานะกับการส่ง message แยก commit กัน · ส่งซ้ำได้เพราะ STA กันซ้ำด้วย message_id)
10	update outbox: outbox_status READY → PUBLISHED → CONFIRMED (บันทึก sent_at) (CONFIRMED ตั้งได้เฉพาะเมื่อได้ publisher confirm จาก broker · Job 10 เฝ้าแถวที่ยังไม่ CONFIRMED ≥ 1 วัน)
11	จบ

2.4 เงื่อนไขตัดสิน (Decision Rules) — ตัดสินจากอะไร

Job 6 มี 2 สวิตช์ที่เปลี่ยนผลลัพธ์ของทั้งรอบ และทั้งคู่ถูกตัดสิน ก่อน เปิด transaction — implement ผิดจะส่งข้อมูลผิดชุดให้ STA โดยไม่มี error ให้เห็น

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
รอบนี้จะสร้างชุด `I` (Initial) ให้ STA หรือไม่	`processDate` (argument · ดู 5.95) เทียบกับค่าคงที่ `dateStartInitToSTA`	dd ของ `processDate` ≥ 7 → สร้างชุด Initial ของงวด (log `Create Data`) · `< 7` → ไม่สร้าง (log `No Create Data`) แต่ยังส่งชุด A/N/S ตามปกติ	ค่านี้อยู่ใน `ApplicationResources.properties` ของระบบเดิม → ระบบใหม่ต้องเป็น env ไม่ใช่ค่าฝังในโค้ด
ข้อมูล QSSI ของงวดครบหรือยัง	`fcs_qssi_score` (ตารางของระบบ SBP เดิม · SGI อ่านอย่างเดียว) — `year` · `month` · `category`	งวดที่ตรวจ = เดือนก่อนหน้า `processDate` · ต้องมีข้อมูลครบทุกหมวดใน 6 หมวด `8, 9, 12, 1, 10, 16` · หมวดใดหมวดหนึ่ง count = 0 → ไม่ครบทันที (ระบบเดิม `break` แล้วตั้ง `countRow = 0`)	ไม่ครบ → ชุดที่ส่งออกเปลี่ยนไป (ไม่ใช่ job fail) · ▲ ระบบเดิม กลืน exception ของคิวรีนี้เป็น 0 ด้วย — ระบบใหม่ต้องแยก "ไม่ครบ" ออกจาก "คิวรีพัง" และ log ต่างกัน
แถวไหนถูกส่งออกและส่งด้วยสถานะอะไร	`sgi_fgi_impact_compensations.compensate_status`	ส่ง A (อนุมัติชดเชย) · N (เห็นควรไม่ชดเชย) · S (หยุดชดเชย) เสมอ · เพิ่ม `I` เมื่อสวิตช์ข้อ 1 และ 2 จริงพร้อมกัน · Z (ยอดเป็นศูนย์) แปลงเป็น S เฉพาะใน payload — ใน DB ยังเป็น Z	R (Reflow) ไม่ได้มาจาก job นี้ — เกิดจากปุ่มเปิดพิจารณาใหม่ที่ `POST /sgi/document/{docNo}/actions` (ดู `LLDD-BE-API-Document-Workflow-Actions.pdf` §5.1c)

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
ปิดรอบชดเชยหรือพักไว้	`sgi_fgi_impact_processes.flag_action`	ส่งผลชดเชยของงวดครบแล้ว → Y → N (ปิดรอบ) · ยังรอจ่ายอีก → Y → W (พัก)	`flag_action` เป็น input ของจุดเขา flow ชั้นที่ 1 ใน Job 8b — เขียนผิดที่นี่ทำให้เอกสารรอบหน้าเข้าผิดขั้น

ช่องว่างที่เคยค้างของหัวข้อนี้ — ปิดครบแล้ว (เก็บไว้เป็นประวัติ)

ทุกข้อปิดแล้ว — เก็บตารางไว้เพื่อให้ตามรอยได้ว่าเคยค้างอะไรและปิดด้วยอะไร

#	สิ่งที่ขาด	ต้องทำอะไรก่อน
G5 ✓ ปิดแล้ว 2026-09-02	`compensate_status VARCHAR(5)` เดิมไม่มีทั้ง CHECK และคำอธิบายโดเมนใน DDL และค่า `'C'` ไม่มีนิยาม	อ่าน `ExportJdbc.insertFgiImpactStoreCompensate()` (บรรทัด 395-460) แล้วพบนิยามของ `'C'` — ดูตาราง "โดเมนของ `compensate_status`" ด้านล่าง · เติมโดเมนครบ + กฎ map ลง DDL แล้ว (`CHECK IN ('I','C','A','N','S','Z')`)

ค่าคงที่และโดเมนที่ใช้ในเงื่อนไขข้างบน

ทุกค่าในตารางนี้ต้องอ่านจาก config/env ไม่ hardcode ในคิวรี — แต่ ค่าตั้งต้นต้องเท่าระบบเดิมทุกตัว ไม่งั้นผลลัพธ์จะไม่ตรงกันตอนเทียบ UAT

ค่า	โดเมน / ค่าที่ระบบเดิมใช้	ที่มา
`dateStartInitToSTA`	`7`	`ApplicationResources.properties` ของระบบเดิม
`categoryQssi`	`8,9,12,1,10,16` (6 หมวด) → 8 = Result · 9 = Process · 12 = สินค้าขาด · 1 = บริการ · 10 = Follow up · 16 = สินค้าหมดอายุ (ลำดับนี้คือลำดับฟิลด์ 9–14 ของ `FRBC0001`)	`ApplicationResources.properties` · ชื่อหมวดถอดได้ 2026-09-16 จากรายงาน `RT040035` ของ STA เทียบกับไฟล์ `FRBC0001` จริงร้านเดียวกันงวดเดียวกัน (ตรงทุกค่า ยืนยัน 3 ร้าน) — ดู `IAS STA interface files` · ▲ เปอร์เซ็นต์หักคำนวณที่ STA ไม่ใช่ที่ SGI SGI ส่งแค่คะแนนดิบ · หมวด Follow up ติดลบได้
`numWaitPay`	3 งวด	`ApplicationResources.properties`
`compensate_status` ที่ STA รับได้	`I` · `A` · `N` · `S` · `R`	ประกันรายได้ ตัวอย่าง Message RabbitMQ §2.2
ค่าที่มีใน DB แต่ไม่ส่งดิบ ๆ	`Z` และ `C` → แปลงเป็น `S` เฉพาะใน payload (ใน DB คงค่าเดิม)	ขั้นที่ 4 ของ Job 6 · `ExportJdbc` จัด C/S/Z เป็นกลุ่มเดียวกันในทุก filter ปลายน้ำ (บรรทัด 366 · 1440)

โดเมนของ `compensate_status` ครบทุกค่า (สืบจาก `FgiConstant` + `ExportJdbc`)

ค่าเหล่านี้ไม่ได้อยู่ใน DDL เดิมเลย · สืบจากโค้ดจริงของระบบเดิมทั้งหมด — `C` เป็นค่าที่ระบบเดิมตั้งตอน insert ไม่ใช่ค่าที่คนเลือก จึงต้องรองรับตั้งแต่ migration รอบแรก

ค่าใน DB	หมายความว่าอะไร	ส่งอะไรให้ STA	ที่มาในโค้ดเดิม
`I`	Initial — ข้อมูลตั้งต้นของงวด (ร้านยังเปิดและสัญญายังไม่จบ)	ส่ง `I`	`FGI_COMPESATE_STATUS_I`
`C`	Closed/Cancelled — ตอน insert แถวงวดนั้นพบว่า ร้านปิดไปแล้ว (mas_store.close_date <= งวด) หรือ สัญญา SBP ถูกยกเลิกก่อน/ในงวดนั้นด้วยเหตุ `cancel_type IN ('01','02','03','04','08')` — ตั้งแทน `I` ตั้งแต่แรก	ส่ง `S` (STA ไม่รับค่า `C`) — กลุ่มเดียวกับ `S`/`Z` ในทุก filter ปลายน้ำ	`ExportJdbc` บรรทัด 404 · 414 · 452 (CASE ตอน insert) · 366 · 1440 (filter)
`A`	Approve — อนุมัติชดเชย	ส่ง `A`	`FGI_COMPESATE_STATUS_A`
`N`	Not Approve — เห็นควรไม่ชดเชย	ส่ง `N`	`FGI_COMPESATE_STATUS_N`
`S`	Stop — หยุดชดเชยประกันรายได้	ส่ง `S`	`FGI_COMPESATE_STATUS_S`

ค่าใน DB	หมายความว่าอะไร	ส่งอะไรให้ STA	ที่มาในโค้ดเดิม
Z	Zero — ยอดชดเชยของงวดเป็นศูนย์	ส่ง S (แปลงเฉพาะใน payload · ใน DB ยังเป็น Z)	`FgiConstant.FLAG_VERIFY_Z`
R	Reflow — เปิดพิจารณาใหม่	เป็นค่าของ message เท่านั้น ไม่เคยลง DB — สร้างโดย `POST /sgi/document/{docNo}/actions`	สเปก STA §2.2 + `LLDD-BE-API-Document-Workflow-Actions.pdf` §5.1c

SQL ตรวจความครบของ QSSI — **ต้องแยกทีละหมวด ห้าม `IN (...)` รวบเดียว**

ระบบเดิมวนตรวจ ทีละหมวด แล้ว break ทันทีที่เจอหมวดว่าง · เขียนเป็น `category IN (8,9,12,1,10,16)` แล้วนับรวมจะได้ผลต่างกัน (มีข้อมูลหมวดเดียวก็ผ่าน) ซึ่ง ผิด และจะส่งชุด Initial ผิดให้ STA

```
-- bind ตามลำดับ: $1=qssi_year · $2=qssi_month · $3=qssi_categories
-- ตรวจทีละหมวด · หมวดใดได้ 0 ให้หยุดทันทีและถือว่า "ไม่ครบ"
SELECT q.category, COUNT(1) AS count_rec
FROM fcs_qssi_score q -- schema sps_store · SGI อ่านอย่างเดียว
WHERE q.year = $1 /* qssi_year */ -- ปีของ "เดือนก่อนหน้า processDate"
  AND q.month = $2 /* qssi_month */ -- เดือนก่อนหน้า processDate
  AND q.category = ANY($3 /* qssi_categories */) -- {8, 9, 12, 1, 10, 16}
GROUP BY q.category;
-- ครบ ก็ต่อเมื่อ จำนวนหมวดที่คืนมา = 6 และทุกหมวด count_rec > 0
-- คิวรี throw = ต้อง log error แล้วหยุดรอบ ห้ามกลืนเป็น "ไม่ครบ" แบบระบบเดิม
```

2.5 Field, Format, and Validation

Field / UI	Format	Validation	Behavior
กำหนดการรัน (Cron)	0 17 * * *	แก้ไขได้	ทุกวัน 17:00
dateStartInitToSTA	7	แก้ไขได้	วันของเดือนที่เริ่มปล่อยสถานะ I, C
numWaitPay	3	แก้ไขได้	จำนวนงวดรอจ่าย
หมวด QSSI ที่ตรวจ	8, 9, 12, 1, 10, 16	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	ต้องครบทั้ง 6 หมวดจากงวด max เดียว ในกรอบ 3 เดือน
RabbitMQ exchange	sgi.interface (topic, durable)	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	อ่านชื่อ exchange จาก backend config (`SGI_MQ_EXCHANGE`) ไม่ hardcode
Routing key	sta.compensation.result	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	STA เป็นเจ้าของ queue ที่ bind มาที่ routing key นี้
Message payload	JSON UTF-8 · dataName = sgi_impact_store (14 ฟิลด์ · สัญญาที่ ประกันรายได้ ตัวอย่าง Message RabbitMQ)	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	ฟิลด์ 3/5/6 คงเป็นวันที่ พ.ศ. ตามสัญญา 14 ฟิลด์เดิมของ STA — แปลงเฉพาะตอนประกอบ payload ห้ามให้ปนเข้า DB/API
Message properties	persistent (delivery_mode=2) · message_id = sgi_interface_transactions.id	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	message_id เป็น idempotency key ให้ STA กันรับซ้ำ
Secret reference	secret/sgi/mq/sta	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	user/password ของ broker จาก Secret Manager · เชื่อมด้วย AMQPS (TLS verify-full)

2.6 หลักฐานที่ต้องเก็บตอนรัน

Evidence	Job-specific value	Acceptance
Input identity	Approved/initial compensation data from FGI impact/new-store tables plus QSSI score lookup and FS export configuration.	บันทึกไฟล์/คีย์ธุรกิจ/งวดที่รับเข้าใน run record
Output identity	FS outbound data and FGI compensation tables synchronized; run summary includes exported counts and file/status.	กระทบยอด input · สำเร็จ · ตก · ข้าม
Dedup proof	UNIQUE(data_name,direction,business_key, period_key); publisher confirm เปลี่ยน outbox_status ของ transaction เดิมเป็น CONFIRMED ไม่ insert แถวใหม่	รันซ้ำด้วย fixture เดิมต้องไม่เกิดคีย์ธุรกิจซ้ำ

Evidence	Job-specific value	Acceptance
Transaction proof	สร้าง payload/checksum ก่อน แล้ว insert outbox READY; dispatcher publish และเปลี่ยน SENT + outbox_status = PUBLISHED แยก transaction; เมื่อได้ publisher confirm จาก broker จึงเปลี่ยน outbox_status = CONFIRMED + status = COMPLETED แบบ compare-and-set บนแถวเดิม (มติ 2026-09-08 ข้อ 2.13 — ไม่มี ACK ระดับธุรกิจจาก STA)	จำลอง error กลางทางแล้วไม่เหลือแถวค้างนอกขอบเขตที่ระบุ
Security proof	RabbitMQ broker ใช้ secretRef=secret/sgi/mq/sta, เชื่อมด้วย AMQPS (TLS 1.2+ verify-full); exchange/routing key มาจาก config ไม่ใช่ค่าที่ผู้ใช้แก้ได้ (dataName = sgi_impact_store ตามสัญญาใน ประกันรายได้ ตัวอย่าง Message RabbitMQ); credential rotation ไม่ต้องแก้เอกสารหรือ job param	config/log/error ไม่มีความลับแบบข้อความธรรมดา

2.7 โค้ด Java เดิมที่อ้างอิง

Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/ExportImpactStoreToFS.java	19-68	Legacy main entrypoint for exporting impact-store compensation to FS.
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java	119-180, 386-970	Query FS export data and insert/update impact/new-store compensation records.

Line ranges refer to the legacy Java implementation under `batchjob/fcsJar/` (path นับจากราก `sbp-prototype/`). Use these ranges to preserve business behavior while implementing the target Node job.

2.8 เขียนข้อมูลรอบชดเชย (รับเข้าโครง 2026-08-21 · gap F8 + F1)

Job 6 คือ job เดียวที่เขียนตารางรอบชดเชยในระบบเดิม — `ExportService.manageDBToFs()` เรียก 5 คำสั่งต่อกันเป็นชุด ระบบใหม่ต้องทำครบเหมือนเดิม แต่เขียนลงตารางของ SGI

ลำดับใน manageDBToFs()	ระบบเดิม (Oracle)	ระบบใหม่ (SGI)	ใช้ทำอะไรต่อ
updateFgiImpactStoreOnProcess(INITDATE)	FGI_IMPACT_STORE_ON_PROCESS · LAST_COMPENSATE_SEQ_NO + 1 เมื่อ FLAG_ACTION='Y' และเพิ่งชดเชยเดือนที่แล้ว	sgi_fgi_impact_processes.last_compensate_seq_no += 1	เคสต่อเนื่อง (SEQ_NO > 1)
insertFgiImpactStoreOnProcess()	แถวใหม่ · LAST_COMPENSATE_SEQ = MAX+1 · SEQ_NO = 1 · FLAG_ACTION='Y' · DATASOURCE	● ในโครงใหม่เป็น UPDATE ไม่ใช่ INSERT — `sgi_fgi_impact_processes` ถูก Job 2 สร้างไว้แล้ว ตอนนำเข้า (`UNIQUE (impacted_store_code, impact_month)`) Job 6 จึงต้อง `UPDATE ... SET flag_action = 'Y'` (จาก `'N'` ที่ Job 2 ใส่) · `last_compensate_seq = MAX+1` · `last_compensate_seq_no = 1` · `start/end_compensate_*` · และเลื่อน `process_status` เป็น `READY_DOCUMENT` — insert ซ้ำจะชนคีย์ (มติ 2026-09-13 · DECISIONS 2.38)	เปิดเรื่องใหม่ (SEQ_NO = 1)
insertFgiImpactStoreCompensate(...)	FGI_IMPACT_STORE_COMPENSATE · COMPENSATE_FORECAST / COMPENSATE_ADJUST ต่องวด	sgi_fgi_impact_compensations (forecast_amount / adjust_amount)	นับยอด 0 ติดกันกี่เดือน (กติกาเดือน 1-3 / เดือนที่ 4)

ลำดับใน manageDBToFs()	ระบบเดิม (Oracle)	ระบบใหม่ (SGI)	ใช้ทำอะไรต่อ
insertFgiNewStoreCompensate(...)	FGI_NEW_STORE_COMPENSATE	sgi_document_new_stores.compensation_amount / compensate_percent	ยอดต่อร้านเปิดใหม่
updateCompleteImpactStoreOnProcess / FlagYToW	FLAG_ACTION Y→N / Y→W	sgi_fgi_impact_processes.flag_action	ปิดรอบ / ส่งกลับรอตรวจ

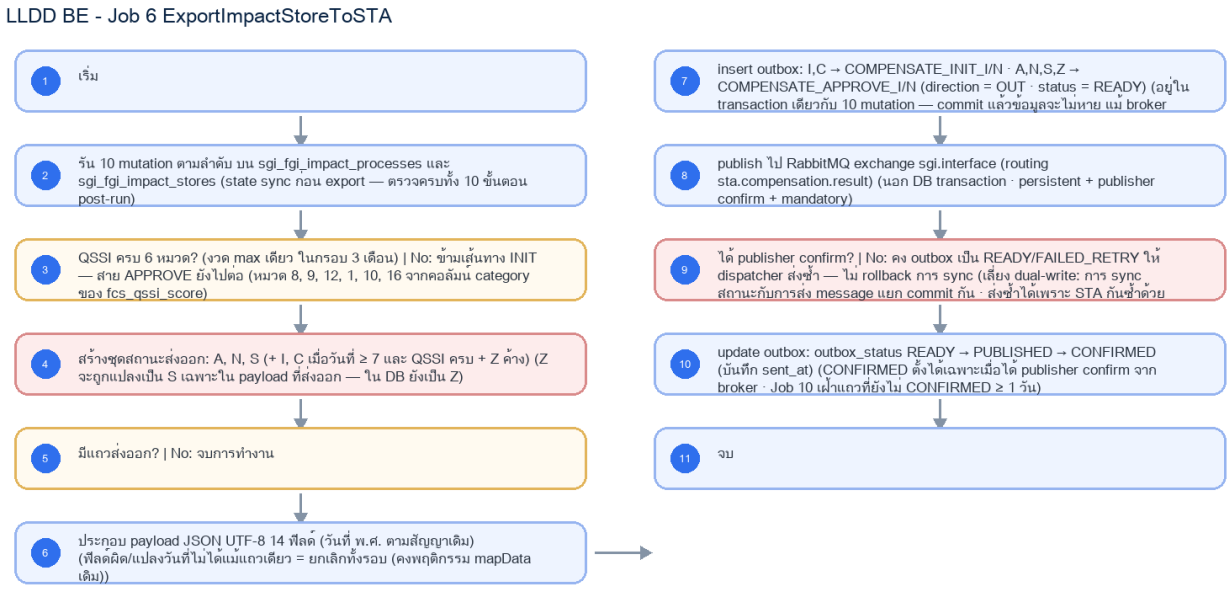
▲ `ImportJdbc.insertImpactStoreOnProcess()` / `updateImpactStoreOnProcess()` มี SQL ชุดเดียวกันอยู่ในไฟล์ Import แต่ตรวจทั้ง src แล้ว ไม่มี call site จริง — เป็นโค้ดตาย ให้ยึด `ExportJdbc` เป็นต้นแบบเท่านั้น

2.9 Tracking Retention / Purge SQL

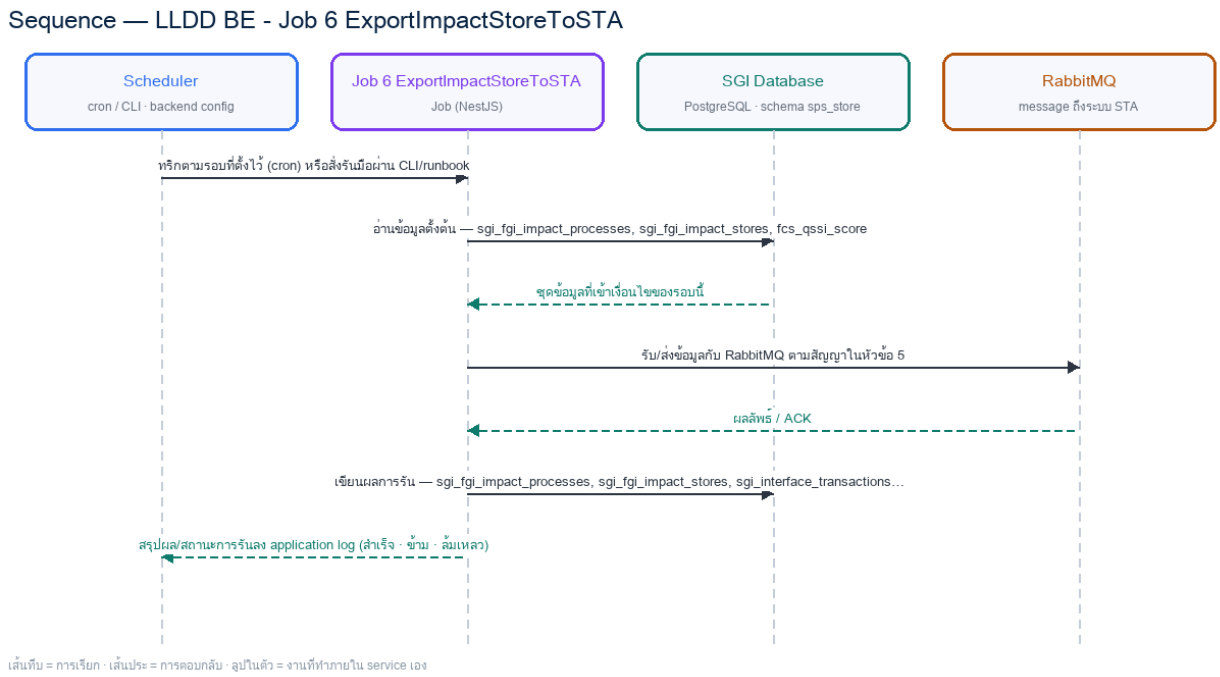
Purge ทำได้เฉพาะแถว COMPLETED ที่ครบ purge_after และไม่อยู่ใน legal hold; ต้องรันเป็น batch จำกัดจำนวนเพื่อไม่ lock ตารางยาว

```
-- bind ตามลำดับ: $1=sta_data_names · $2=batch_size
WITH purge_candidates AS (
    SELECT id
    FROM sgi_interface_transactions
    WHERE status = 'COMPLETED'
      AND purge_after < CURRENT_TIMESTAMP
      AND legal_hold = FALSE
      AND data_name = ANY($1 /* sta_data_names */)
    ORDER BY id
    LIMIT $2 /* batch_size */
    FOR UPDATE SKIP LOCKED
)
DELETE FROM sgi_interface_transactions i
USING purge_candidates p
WHERE i.id = p.id
RETURNING i.id, i.data_name, i.business_key;
```

3. Flow และ Sequence Diagram



รูปที่ 1: Implementation flow: LLDD BE - Job 6 ExportImpactStoreToSTA



รูปที่ 2: Sequence diagram: LLDD BE - Job 6 ExportImpactStoreToSTA

4. Reference DB Mapping

อ้างอิงสำหรับ implement job เท่านั้น ไม่ใช่งานออกแบบ/สร้างหน้า Database

Table / Object	R/W	Usage
sgi_fgi_impact_processes	R/W	หนึ่งใน 10 mutation (สถานะ process / last_compensation_amount)
sgi_fgi_impact_stores	R/W	สถานะค่าชดเชย I/C/A/N/S/Z และข้อมูลร้าน/ผู้อนุมัติ/ค่าชดเชยร้านใหม่
fcs_qssi_score	R	ตรวจความครบคะแนน 6 หมวด — อ่านอย่างเดียว ระบบ SBP เดิมเป็นคนนำเข้า (คอลัมน์จริง: store_id · category · month · year · score)
sgi_fgi_impact_sales_summaries	R	growth_rate_diff / total_working_days ที่ใช้คัดชุดส่งออก (เพิ่ม 2026-09-02 — SQL แตะอยู่แล้วแต่ไม่ได้ประกาศไว้)
sgi_compensation_documents	R	ผลพิจารณาของงวดที่ต้องส่งไป STA (เพิ่ม 2026-09-02 — SQL แตะอยู่แล้วแต่ไม่ได้ประกาศไว้)
sgi_consideration_logs	R	ผลการพิจารณาล่าสุดของเอกสาร ใช้ประกอบ compensate_status (เพิ่ม 2026-09-02 — SQL แตะอยู่แล้วแต่ไม่ได้ประกาศไว้)
sgi_interface_transactions	W	outbox + tracking COMPENSATE_INIT / APPROVE (I,N) · direction = OUT · READY → PUBLISHED → CONFIRMED · typed FK = impact_process_id

5. โค้ดจริงอยู่ที่ไหน

เอกสารนี้ ไม่วาง skeleton code — job ทั้ง 11 ตัว implement แล้ว การมีโค้ดสองชุด (ในเอกสาร + ของจริง) ทำให้หลุดจากกันแน่นอน · อ่านสัญญาที่หัวข้อ 2 แล้วดูของจริงตามตารางนี้

สิ่งที่ต้องดู	ที่อยู่
ชื่อ job ที่ลงทะเบียน	`sgi-job-6` ใน `SBP/srm-sps-spsap-sop-sgi-batch/src/main.ts`

สิ่งที่ต้องดู	ที่อยู่
service ของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-6-*.service.ts`
เทสของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-6-*.spec.ts`
SQL ที่ใช้จริง	อยู่ในไฟล์ service เดียวกัน (repo นี้ไม่แยก `*.sql.ts`)
คำอธิบายภาษาคนของ job	batchjob/JOB-06-* อธิบายละเอียด

6. Acceptance Criteria

- พารามิเตอร์รับผ่าน `INPUT` (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ `@Cron` ในโค้ด
- การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนพึ่ง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้
- ทุกรอบต้องเขียน application log แบบ structured (`BATCH_START`/`BATCH_END` + `runId` — `src/main.ts` ทำให้แล้ว) และบันทึกลง `integration_log` อัตโนมัติ · error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

7. Unit Test Case

63 เคส · ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง (mock ที่ขอบ repository/client) · ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่ · ● เทสที่ mock query runner ต้องป้อนรูป tuple `[rows, rowCount]` สำหรับ UPDATE/DELETE (กฎ #140)

การรับ Argument

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-01	ไม่ส่ง argument	เรียก batch โดยไม่มี argument	ใช้วันที่ปัจจุบันเป็นวันที่อ้างอิง ทำงานต่อปกติ
UT-02	ส่ง argument เป็นค่าว่าง	argument เป็น string ว่าง	ใช้วันที่ปัจจุบันเป็นวันที่อ้างอิง เสมือนไม่ได้ส่ง argument
UT-03	ส่ง argument ที่ parse เป็นวันที่ได้ถูกต้อง	`"20260615"`	ใช้ `2026-06-15` เป็นวันที่อ้างอิง
UT-04	ส่ง argument ที่ parse ไม่สำเร็จ	`"invalid-date"`	บันทึก error message, ไม่เรียกตัวประมวลผลหลัก, ผลลัพธ์เป็น FAILED, exit code = 1

การคำนวณเงื่อนไขวันที่และการตรวจสอบ QSSI

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-05	วันที่อ้างอิงอยู่ในช่วงวันเกณฑ์ถึงวันสุดท้ายของเดือน	วันของวันที่อ้างอิง >= dateStartInitToSTA และ <= วันสุดท้ายเดือนนั้น	`isInitPeriodWindow = true`
UT-06	วันที่อ้างอิงอยู่นอกช่วงวันเกณฑ์	วันของวันที่อ้างอิง < dateStartInitToSTA	`isInitPeriodWindow = false`
UT-07	วันของวันที่อ้างอิง >= วันเกณฑ์	เช่น dateStartInitToSTA=7, วันที่อ้างอิงวันที่ 10	`isRolloverDay = true`
UT-08	วันของวันที่อ้างอิง < วันเกณฑ์	เช่น dateStartInitToSTA=7, วันที่อ้างอิงวันที่ 3	`isRolloverDay = false`
UT-09	ทุก category ใน config มีคะแนน QSSI อย่างน้อย 1 แถวของงวดอ้างอิง	mock ทุก category คืนค่า count > 0	`isQssiComplete = true`

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-10	มี category ใดหนึ่งไม่มีคะแนน QSSI ของงวดอ้างอิง	mock category แรกคืนค่า count = 0	`isQssiComplete = false` หยุดตรวจ category ที่เหลือ
UT-11	เกิด error จากฐานข้อมูลระหว่างตรวจ QSSI	mock query error ที่ category ใดหนึ่ง	`isQssiComplete = false` ไม่ throw error, ทำงานต่อปกติ

การปรับปรุงสถานะกระบวนการค้างเก่า

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-12	กระบวนการค้างสถานะรอเกิน 30 วัน โดยไม่มีการยืนยันรับข้อมูลกลับมา และไม่มีสถานะ 'S'	กระบวนการที่ `flag_action='W'`, `update_date` เกิน 30 วัน, สถานะค่าชดเชยล่าสุด N ตัวติดกันเป็น 'Z'	`flag_action` เปลี่ยนเป็น `'N'`
UT-13	▲ กระบวนการ active ที่มีรายการค่าชดเชยสถานะ `D`	กระบวนการ `flag_action='Y'` มีรายการสถานะ `D`	`flag_action` เปลี่ยนเป็น `'N'` · ● แต่ `D` ยังไม่อยู่ในโดเมน CHECK ของ DDL ใหม่ (รับ `A/C/I/N/S/Z`) ขณะที่ Oracle ของจริงมี `D` 137 แถว และ `G` 3 แถว → ต้องปิด ข้อค้าง 2.47 ก่อน ไม่งั้นเคสนี้ทดสอบไม่ได้
UT-14	กระบวนการ active ที่มีรายการเริ่มต้นส่งไปแล้ว 3 ครั้งไม่มีการยืนยัน	กระบวนการ `flag_action='Y'`, compensate_status='I' ส่ง COMPENSATE_INIT_I 3 ครั้งไม่มี return_code	`flag_action` เปลี่ยนเป็น `'N'`
UT-15	กระบวนการ active ที่งวดล่าสุดจบแล้วและงวดถัดไปยังไม่เริ่ม	กระบวนการ `flag_action='Y'` ไม่มีกระบวนการลูกที่เริ่มงวดถัดไป	`flag_action` เปลี่ยนเป็น `'W'`, ตั้ง `end_compensate_month/year` เป็นงวดล่าสุด
UT-16	กระบวนการ active ที่มีกระบวนการลูกที่ต่องวดแล้วโดยรายการค่าชดเชยปิด (`'C'`)	กระบวนการลูกมี compensate_status='C'	`flag_action` เปลี่ยนเป็น `'W'`

การสร้างข้อมูลงวดใหม่ (เมื่อ isRolloverDay=true)

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-17	isRolloverDay = false	วันของวันที่อ้างอิง < dateStartInitToSTA	ข้ามขั้นตอนสร้างงวดใหม่ทั้งหมด (ขั้นตอนที่ 8-11)
UT-18	กระบวนการ active ที่งวดล่าสุดตรงกับ (งวดอ้างอิง-2เดือน)	กระบวนการ `flag_action='Y'`, `last_compensate_month/year` = อ้างอิง-2เดือน	เลื่อนงวดเป็นอ้างอิง-1เดือน, `last_compensate_seq_no+1`
UT-19	ร้านที่ถูกกระทบผ่านการตรวจสอบยอดขายแล้วและยังไม่มีกระบวนการครอบคลุมงวดนี้	`sgi_fgi_impact_stores.verify_status='P'`, `sgi_fgi_impact_sales_summaries.sales_status='Y'`	สร้างกระบวนการใหม่ `flag_action='Y'`
UT-20	ร้านที่มาจากแหล่งข้อมูล Franchise Statement (STA)	`fis.datasource='STA'`	สร้างกระบวนการใหม่แม้ไม่มีผลตรวจยอดขาย ALLMAP
UT-21	ร้านปิดตัวหรือถูกยกเลิกแฟรนไชส์ภายในงวดที่จะสร้างรายการค่าชดเชย	`mas_store.close_date` หรือ `cancel_type` ตรงเงื่อนไข	สร้างรายการค่าชดเชยด้วยสถานะ `'C'`
UT-22	ร้านยังเปิดอยู่ปกติ	ไม่ตรงเงื่อนไขปิด/ยกเลิก	สร้างรายการค่าชดเชยด้วยสถานะ `'I'`, ฟิลด์พยากรณ์/ผู้สร้างเป็นค่าว่างเปล่า
UT-23	ร้านเปิดใหม่ที่เกี่ยวข้องยังไม่ปิดตัวในงวดนี้	`ms.close_date` เป็นค่าว่างหรือมากกว่าวันที่ในงวด	สร้างรายการค่าชดเชยร้านเปิดใหม่คู่กับขั้นตอนก่อนหน้า
UT-24	สร้างข้อมูลสรุปร้านที่ถูกกระทบจากแหล่งข้อมูล STA	`fis.datasource='STA'`	`impact_type='04'`
UT-25	สร้างข้อมูลสรุปร้านที่ถูกกระทบที่ผ่านตรวจยอดขายแต่จำนวนวันทำการผิดปกติ	ผลตรวจยอดขายผ่านแต่ working day ผิดปกติหรือ growth rate ไม่มีค่า	`impact_type='03'`

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-26	สร้างข้อมูลสรุปร้านที่ถูกกระทบที่ผ่านตรวจยอดขายครบ 60 วันและเปิดหลัง 1 ต.ค. 2557	ผลตรวจยอดขายผ่านครบเงื่อนไข	`impact_type='02'`
UT-27	สร้างข้อมูลสรุปร้านที่ไม่เข้าเงื่อนไขข้างต้น	กรณีอื่นที่เหลือ	`impact_type='01'`
UT-28	สร้างข้อมูลสรุปร้านจากแหล่งข้อมูล ALM ที่ตั้งค่า URL แผนที่	`fis.datasource='ALM'` และมี url_embedded	สร้างลิงก์แผนที่แบบฝัง (base64) เก็บใน `url_all_map`
UT-29	ปรับสถานะร้านที่ถูกกระทบเป็นผ่านหลังสร้างข้อมูลสรุปครบ	มีข้อมูลใน `sgi_document_new_stores+sgi_fgi_impact_processes` ตรงกัน	`sgi_fgi_impact_stores.verify_status` เปลี่ยนจาก `'P'` เป็น `'Y'`
UT-30	ปรับสถานะร้านที่ถูกกระทบเป็นไม่ผ่านตามผลตรวจยอดขาย	`sgi_fgi_impact_sales_summaries.sales_status='N'`	`sgi_fgi_impact_stores.verify_status` เปลี่ยนจาก `'P'` เป็น `'N'`
UT-31	เกิด error จากฐานข้อมูลระหว่างขั้นตอนที่ 6-14 ข้อใดข้อหนึ่ง	mock DB error ที่ขั้นตอนสร้างข้อมูลสรุปร้าน	rollback ทั้งหมดในรอบนี้, ผลลัพธ์ FAILED, ไม่ทำขั้นตอนดึงข้อมูล export ต่อ

การดึงข้อมูลที่พร้อม Export

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-32	isInitPeriodWindow=true และ isQssiComplete=true	ทั้งสองเงื่อนไขเป็นจริง	รวมสถานะค่าชดเชย `'I','C','A','N','S'` เข้า filter
UT-33	isInitPeriodWindow=false หรือ isQssiComplete=false	เงื่อนไขใดเงื่อนไขหนึ่งเป็นเท็จ	รวมเฉพาะสถานะค่าชดเชย `'A','N','S'` เข้า filter
UT-34	มีรายการค่าชดเชยสถานะ `'Z'` ที่ค้างเกิน 30 วันไม่มีสถานะ `'S'` ในกระบวนการเดียวกัน	ตรงเงื่อนไข window function ตรวจค้าง	รวมรายการนั้นเข้าผลลัพธ์เพิ่มเติมนอกจาก filter สถานะปกติ
UT-35	รายการที่ถูกส่งออกไปแล้ว (มีประวัติ COMPENSATE_INIT_I/APPROVE_I ตรงกันแล้ว)	มี `sgi_interface_transactions` ตรงกับ compensate_i_id นั้น	ไม่รวมรายการนั้นในผลลัพธ์ (กันส่งซ้ำ)
UT-36	ไม่มี QSSI ครบเกณฑ์และถึงรอบสร้างงวดใหม่และรายการว่าง	isQssiComplete=false, isRolloverDay=true, ผลลัพธ์ query ว่าง	note="ไม่พบข้อมูลคะแนน QSSI" (ตามเจตนาเดิม), ผลลัพธ์ NO_DATA
UT-37	รายการว่างกรณีทั่วไป	ผลลัพธ์ query ว่าง โดยไม่เข้าเงื่อนไข UT-36	note="ไม่พบข้อมูล", ผลลัพธ์ NO_DATA
UT-38	มีรายการอย่างน้อยหนึ่งรายการ	ผลลัพธ์ query ไม่ว่าง	ทำขั้นตอนสร้างไฟล์ต่อ

การสร้างข้อความและ publish เข้า RabbitMQ

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-39	ประกอบซองข้อความ	รายการค่าชดเชยที่พร้อมส่ง 3 แถว	ได้ envelope `{dataType:'message', dataName:'sgi_impact_store', dataMessage:[...3], sender, sentAt}` เป็น JSON UTF-8 · ไม่สร้างไฟล์ใด ๆ (มติ 2026-08-24 — เลิกไฟล์ FRBC0001 + SFTP)
UT-46	publish สำเร็จและได้ publisher confirm จาก broker	broker ตอบ confirm	ตั้ง `sgi_interface_transactions.outbox_status = 'CONFIRMED'` · ผลลัพธ์ SUCCESS

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-47	• publish ไม่สำเร็จ / ไม่ได้ confirm	broker ตัดการเชื่อมต่อระหว่าง publish	ข้อมูลที่ commit ไปแล้ว ต้องไม่ rollback (publish อยู่นอก transaction) · แถว outbox คงสถานะ ยังไม่ confirm เพื่อให้ Job 10 เก็บตก · ห้ามตั้ง `CONFIRMED`

การประกอบฟิลด์ของข้อความ

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-40	มีวันที่เปิดร้านใหม่	record ที่มี `opendate_n`	`opendate_n` เป็น `dd/MM/yyyy` ปี พ.ศ. (เช่น `01/09/2567`) — ข้อยกเว้นที่ `API design` ระบุไว้ · แปลงเฉพาะตอนประกอบ payload ห้ามให้ พ.ศ. ปนเข้า DB/API ของ SGI
UT-41	ไม่มีวันที่เปิดร้านใหม่	`opendate_n` ว่าง	ส่งฟิลด์เป็นค่าว่าง ไม่ใช่ `null` string และไม่ตัดฟิลด์ทิ้ง
UT-42	มีงวด statement ครบทั้งเดือนและปี	`stmt_month` + `stmt_year`	รวมเป็น `stmt_year_month` รูปแบบ `yyMM` พ.ศ. (เช่น `6709`) — JSON จริงรวมสองฟิลด์ ไม่ได้แยกเหมือนสเปกรุ่นไฟล์
UT-43	งวด statement ไม่ครบ	`stmt_month` หรือ `stmt_year` ว่างอย่างใดอย่างหนึ่ง	`stmt_year_month` เว้นว่าง
UT-44	คะแนน QSSI ที่ไม่มีค่า	qssi score เป็นค่าว่าง	ส่งเป็นข้อความว่าง ไม่ใช่เลข 0 (0 มีความหมายว่าได้คะแนน 0)
UT-45	• การเข้ารหัสอักขระ	ข้อมูลที่มีตัวอักษรไทย	payload เป็น JSON UTF-8 · ไม่มี windows-874 และไม่มีตัวคั่น `|` แล้ว (ของเดิมเป็นไฟล์ text คั่น `|` + windows-874)

การบันทึกแถว outbox

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-48	• คอลัมน์ที่ใช้ระบุชนิดข้อความ	รายการที่ส่งออกทั้งฝั่งร้านถูกกระทบและร้านเปิดใหม่	บันทึก `sgi_interface_transactions.data_name` เป็นค่าใน enum (`COMPENSATE_INIT_I/_N` · `COMPENSATE_APPROVE_I/_N`) · ✗ ห้ามเก็บชื่อไฟล์ลงคอลัมน์นี้ — ของเดิมเอา `interface_type` ไปเก็บชื่อไฟล์ export ซึ่งขัดกับชื่อคอลัมน์ตัวเอง (ข้อสังเกตจากทีม fgi)
UT-50	ฐานข้อมูลล้มระหว่าง insert แถว outbox	mock DB error	rollback transaction ทั้งหมด · ยังไม่ publish อะไรออกไป (publish อยู่หลัง commit) · ผลลัพธ์ `FAILED`

การบันทึกประวัติการส่งข้อมูลและสำรองไฟล์

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-49	insert ประวัติที่ซ้ำกับที่มีอยู่แล้ว	มี record ตรงกับ data_name+transaction_pk+month+year อยู่แล้ว	ไม่ insert ซ้ำ (กันซ้ำด้วย NOT EXISTS)

ลำดับ transaction กับการ publish

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-51	• ลำดับ commit ก่อน publish	รอบที่มีทั้ง sync สถานะและ outbox	DB transaction ครอบ sync + เขียนแถว outbox แล้ว commit ก่อน จึง publish — ห้าม publish ขณะ transaction ยังเปิด (transactional outbox)
UT-52	รันซ้ำรอบเดิม (idempotent)	รันรอบเดิมอีกครั้งหลังสำเร็จ	ไม่สร้างแถว outbox ซ้ำ และไม่ publish ข้อความเดิมซ้ำ

การลบประวัติ/log ที่เก่าเกินกำหนด

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-53	มีประวัติการส่งข้อมูลของกระบวนการที่ถูกยกเลิกครบ 24 เดือน	กระบวนการ `flag_action='N'` ที่ถึงวันครบกำหนด 24 เดือนหลังงวดสุดท้าย	ลบประวัติที่เกี่ยวข้องทั้ง 6 กลุ่มข้อมูล
UT-54	ไม่มีประวัติที่ต้องลบ	ไม่มีกระบวนการที่ถึงเงื่อนไข	ไม่มีแถวถูกลบ ไม่มี error
UT-55	มี log web service ที่เก่าเกิน 24 เดือน	`integration_log.create_date` เก่ากว่า 24 เดือน	ลบ log นั้นโดยไม่กรองตามชื่อ web service
UT-56	เกิด error จากฐานข้อมูลระหว่าง sub-flow ลบประวัติ/log	mock DB error ใน transaction แยกนี้	rollback เฉพาะ transaction แยกนี้, ไม่กระทบผลลัพธ์หลักที่ commit ไปแล้ว, ผลลัพธ์หลักไม่กลายเป็น FAILED
UT-57	sub-flow นี้ถูกเรียกทั้งกรณี NO_DATA และกรณี SUCCESS	ทั้งสองเส้นทาง (ไม่ error ใน transaction หลัก)	sub-flow ทำงานทุกครั้ง ไม่ว่าจะมีข้อมูล export หรือไม่

ผลลัพธ์มาตรฐานและ Error

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-58	ผลลัพธ์เป็นสำเร็จหรือไม่มีข้อมูล	ทำงานจบปกติ	exit code = `0`
UT-59	ผลลัพธ์เป็นล้มเหลว	เกิด error ณ จุดใดก็ตาม	exit code = `1`
UT-60	error message ที่แสดงใน result ต้อง sanitize	error ที่มี connection string/credential อยู่ใน stack trace เดิม	ข้อความ error ที่แสดงไม่มีข้อมูลอ่อนไหวหลุดออกมา

การแจ้งเตือน / Notification

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-61	หา template และส่งอีเมลสำเร็จ	mock ทั้งสองขั้นตอนสำเร็จ	อีเมลถูกส่งด้วยเนื้อความที่ render จากผลลัพธ์จริง ไปยังผู้รับที่ตั้งค่าไว้ใน config โดยใช้ที่อยู่ผู้ส่งหลัก
UT-62	หา template ไม่พบ	mock query template คืนค่าว่าง	log ไว้เท่านั้น ไม่ throw error, ไม่กระทบผลลัพธ์ที่ commit แล้ว
UT-63	ส่งอีเมลผ่าน SMTP ล้มเหลว	mock SMTP ล้มเหลว	log ไว้เท่านั้น, ผลลัพธ์ที่ได้ก่อนหน้าไม่เปลี่ยนเป็นล้มเหลว, ไม่มีการ retry

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. ExportImpactStoreToFS.java — lines 19-30

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ExportImpactStoreToFS.java
Original lines	19-30
Responsibility	Legacy main entrypoint for exporting impact-store compensation to FS.

```
public class ExportImpactStoreToFS {
private static ApplicationContext context = new FileSystemXmlApplicationContext(
"config.xml");
private static ExportController exportController = (ExportController) context.getBean("exportController");
private static final FgiConfig config = (FgiConfig) context.getBean("fgiConfig");

public static void main(String[] args){
String parameter = null;
boolean isParameter=false, isCompleted=false, isCreateData=false;
Calendar StartDateTime = Calendar.getInstance(Locale.US);
final EmailTemplateInformStatusBean informBean = new EmailTemplateInformStatusBean();
informBean.setSystemCode(FgiConstant.SYSTEM_CODE);
```

J1. ExportImpactStoreToFS.java — lines 57-68

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ExportImpactStoreToFS.java
Original lines	57-68
Responsibility	Legacy main entrypoint for exporting impact-store compensation to FS.

```
}else{
informBean.setStatus(FgiConstant.JOB_STATUS_SUCCESS);
}
SendMailUtil.sendMailToAdmin(informBean, config, true);
}

// String endTime = DateUtils.getCurrentDate(DateUtils.LOCALE_EN,FgiConstant.DATE_FORMAT_DDMMYYYY_TIME)+"";
LogUtils.info(ExportImpactStoreToFS.class,"End => export store compensate to STA");
}

}
```

J2. ExportJdbc.java — lines 119-130

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	119-130
Responsibility	Query FS export data and insert/update impact/new-store compensation records.

```
public List<Map<String, String>> queryFmlImpactStoreToFS(boolean isQssi, boolean isCreateInitToSTA, final String NUM_WAIT_PAY, final String INITDATE){
StringBuffer sql = new StringBuffer();
String compensate_status = "'"+FgiConstant.FGI_COMPESATE_STATUS_A+"', '"+FgiConstant.FGI_COMPESATE_STATUS_N+"',
'"+FgiConstant.FGI_COMPESATE_STATUS_S+"'";
if(isCreateInitToSTA&&isQssi){
compensate_status="'"+FgiConstant.FGI_COMPESATE_STATUS_I+"', '"+FgiConstant.FGI_COMPESATE_STATUS_C+"',
'"+FgiConstant.FGI_COMPESATE_STATUS_A+"', '"+FgiConstant.FGI_COMPESATE_STATUS_N+"', '"+FgiConstant.FGI_COMPESATE_STATUS_S+"'";
}

sql.append(" select fisi.storecode_i, fnsi.storecode_n, fnsi.opendate_n, ");
sql.append(" count(fnsc.storecode_n) over(partition by fnsc.impact_process_id, fnsc.compensate_month, fnsc.compensate_year) as count_storecode_n, ");
sql.append(" fisc.compensate_month, fisc.compensate_year, ");
sql.append(" fisc.stmt_month, fisc.stmt_year, ");
sql.append(" decode(fisc.compensate_status, '"+FgiConstant.FLAG_VERIFY_Z+"', '"+FgiConstant.FGI_COMPESATE_STATUS_S+"', fisc.compensate_status) as
compensate_status, ");
```

J2. ExportJdbc.java — lines 169-180

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	169-180
Responsibility	Query FS export data and insert/update impact/new-store compensation records.

```
sql.append(" left join fcs_qssi_score qs1 on qs1.store_id = qs.store_id ");
sql.append(" and to_date(qs1.year || qs1.month, 'YYYYMM') = qs.qssi_period ");
sql.append(" and qs1.category = 8 ");
sql.append(" left join fcs_qssi_score qs2 on qs2.store_id = qs.store_id ");
sql.append(" and to_date(qs2.year || qs2.month, 'YYYYMM') = qs.qssi_period ");
sql.append(" and qs2.category = 9 ");
sql.append(" left join fcs_qssi_score qs3 on qs3.store_id = qs.store_id ");
sql.append(" and to_date(qs3.year || qs3.month, 'YYYYMM') = qs.qssi_period ");
sql.append(" and qs3.category = 12 ");
sql.append(" left join fcs_qssi_score qs4 on qs4.store_id = qs.store_id ");
sql.append(" and to_date(qs4.year || qs4.month, 'YYYYMM') = qs.qssi_period ");
sql.append(" and qs4.category = 1 ");
```

J2. ExportJdbc.java — lines 386-397

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	386-397
Responsibility	Query FS export data and insert/update impact/new-store compensation records.

```
public int insertFgiImpactStoreCompensate(final List<ImpactStore> impactStoreList, final String flag){
int result=0;
String sqlAppend = "";
if(FgiConstant.FRANCHISE_STATEMENT.equalsIgnoreCase(flag)){
sqlAppend = " and op.storecode_i = ? "
+" and op.last_compensate_month = ? "
+" and op.last_compensate_year = ? ";
}
StringBuilder sql = new StringBuilder();
sql.append(" insert into fgi_impact_store_compensate ( ");
sql.append(" compensate_i_id, impact_process_id, storecode_i, compensate_seq, compensate_seq_no, ");
sql.append(" compensate_month, compensate_year, compensate_forecast, compensate_adjust, compensate_status, ");
```

J2. ExportJdbc.java — lines 959-970

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	959-970
Responsibility	Query FS export data and insert/update impact/new-store compensation records.

```
sql.append(" decode(op.last_compensate_month || op.last_compensate_year, fis.month || fis.year, fis.opendate_i, ms.open_date) as opendate_i, ");
sql.append(" ms.close_date as closedate_i, ");
sql.append(" decode(op.last_compensate_month || op.last_compensate_year, fis.month || fis.year, fis.zone_i, ms.region) as zone_i, ");
sql.append(" decode(op.last_compensate_month || op.last_compensate_year, fis.month || fis.year, fis.branchtype_i, ms.status_type) as branchtype_i, ");
sql.append(" decode(op.last_compensate_month || op.last_compensate_year, fis.month || fis.year, fis.juristic_id_i, j.juristic_id) as juristic_id_i, ");
sql.append(" decode(op.last_compensate_month || op.last_compensate_year, fis.month || fis.year, fis.juristic_name_i, j.juristic_name) as juristic_name_i, ");
sql.append(" fz.first_name as franchisee_fname_i, fz.last_name as franchisee_lname_i, ");
sql.append(" case when (op.datasource = '"+FgiConstant.FRANCHISE_STATEMENT+"') then ");
sql.append(" '04' ");
sql.append(" when (fiss.flag_verify = '"+FgiConstant.FLAG_VERIFY_ACCEPT+"' and (fiss.total_working_day != "+FgiConstant.TOTAL_INTERVAL_DAY+" or
fiss.growth_rate_diff is null)) then ");
sql.append(" '03' ");
sql.append(" when ( ");
```